

BUS BOOKING SYSTEM

FRONTEND :

Data Binding :

- Signals are being used in handling state changes and modifying UI as per the change.
- Signals is used for majority of data binding throughout the frontend.

```
export class SearchPageComponent implements OnInit {
  readonly loading = signal(false);
  readonly error = signal('');
  readonly searched = signal(false);
  readonly buses = signal<BusSearchResult[]>([]);
  readonly places = signal<string[]>([]);
  readonly showGuardModal = signal(false);
  readonly guardMode = signal<'LOGIN' | 'ROLE_DENIED'>('LOGIN');

  private readonly fb = inject(FormBuilder);

  readonly form = this.fb.nonNullable.group({
    source: ['', [Validators.required]],
    destination: ['', [Validators.required]],
    date: [new Date().toISOString().split('T')[0], [Validators.required]]
  });
```

- At some places while rendering the data fetched from the server interpolation type of binding is used.

```
<p class="mt-2 text-sm text-slate-300">{{ bus.operatorName }} · {{
bus.registrationNumber }}</p>
    <p class="mt-1 text-sm text-slate-400">{{ bus.travelDate }} · {{
bus.departureTime }} - {{ bus.arrivalTime }}</p>
    <p class="mt-3 text-lg font-semibold text-white">₹{{ bus.seatPrice
}}</p>
```

Folder Structure :

- There is no component based architecture
- No testing files were created ie the [component.spec.ts](#) files and no jasmine and karma integration.
- The models were not divided into separate files.
- All the models were coded in a single file called [models.ts](#)

```
export type Role = 'USER' | 'BUS_OPERATOR' | 'ADMIN';

export interface ApiResponse<T> {
  success: boolean;
  data: T;
  message?: string;
  errors?: string[];
}

export interface AuthPayload {
  userId: string;
  username: string;
  email: string;
  role: Role;
  token: string;
  expiresAtUtc: string;
}

export interface Profile {
  userId: string;
  username: string;
  email: string;
  role: Role;
}
```

And many more models....

- Only one test file ie [app.spec.cs](#) that too only checks whether the app run:

```
it('should create the app', () => {
  const fixture = TestBed.createComponent(App);
  const app = fixture.componentInstance;
  expect(app).toBeTruthy();
});
```

- The components were not used by importing in the app.ts ,instead they are rendered using RoutingOutlet as shown below.

```
export const routes: Routes = [
  { path: 'login', component: LoginPageComponent },
  { path: 'signup', component: SignupPageComponent },
  {
    path: '',
    component: ShellComponent,
    children: [
      { path: '', pathMatch: 'full', redirectTo: 'search' },
      { path: 'search', component: SearchPageComponent },
      { path: 'book/:scheduleId', component: BookBusPageComponent, canActivate:
[authGuard, roleGuard], data: { roles: ['USER', 'BUS_OPERATOR', 'ADMIN'] } },
      { path: 'my-bookings', component: MyBookingsPageComponent, canActivate:
[authGuard, roleGuard], data: { roles: ['USER'] } },
      { path: 'operator-request', component: OperatorRequestPageComponent,
canActivate: [authGuard, roleGuard], data: { roles: ['USER'] } },
      { path: 'operator/dashboard', component: OperatorDashboardPageComponent,
canActivate: [authGuard, roleGuard], data: { roles: ['BUS_OPERATOR'] } },
      { path: 'operator/routes', component: OperatorRoutesPageComponent,
canActivate: [authGuard, roleGuard], data: { roles: ['BUS_OPERATOR'] } },
      { path: 'operator/buses', component: OperatorBusesPageComponent,
canActivate: [authGuard, roleGuard], data: { roles: ['BUS_OPERATOR'] } },
      { path: 'admin/dashboard', component: AdminDashboardPageComponent,
canActivate: [authGuard, roleGuard], data: { roles: ['ADMIN'] } },
      { path: 'admin/routes', component: AdminRoutesPageComponent, canActivate:
[authGuard, roleGuard], data: { roles: ['ADMIN'] } },
      { path: 'admin/operator-requests', component:
AdminOperatorRequestsPageComponent, canActivate: [authGuard, roleGuard], data:
{ roles: ['ADMIN'] } },
      { path: 'admin/config', component: AdminConfigPageComponent, canActivate:
[authGuard, roleGuard], data: { roles: ['ADMIN'] } },
      { path: 'admin/bookings', component: AdminBookingsPageComponent,
canActivate: [authGuard, roleGuard], data: { roles: ['ADMIN'] } }
    ]
  },
  { path: '**', redirectTo: '' }
];
```

- So for each routes the corresponding component is mapped.
- The environment variables were stored in a separate file in environment folder inside app directory.

Coding Practices :

- Use of @if used for conditional rendering like admin dashboard, operator dashboard etc.. as shown below

```
@if (isUser()) {
    <a routerLink="/my-bookings" routerLinkActive="bg-slate-700 text-white"
class="rounded px-3 py-1.5 text-slate-300 hover:bg-slate-800">My Bookings</a>
    <a routerLink="/operator-request" routerLinkActive="bg-slate-700
text-white" class="rounded px-3 py-1.5 text-slate-300 hover:bg-slate-800">Become
Operator</a>
}
@if (isOperator()) {
    <a routerLink="/operator/dashboard" routerLinkActive="bg-slate-700
text-white" class="rounded px-3 py-1.5 text-slate-300 hover:bg-slate-800">Operator</a>
    <a routerLink="/operator/routes" routerLinkActive="bg-slate-700
text-white" class="rounded px-3 py-1.5 text-slate-300 hover:bg-slate-800">Routes</a>
    <a routerLink="/operator/buses" routerLinkActive="bg-slate-700
text-white" class="rounded px-3 py-1.5 text-slate-300 hover:bg-slate-800">Buses</a>
}
@if (isAdmin()) {
    <a routerLink="/admin/dashboard" routerLinkActive="bg-slate-700
text-white" class="rounded px-3 py-1.5 text-slate-300 hover:bg-slate-800">Admin</a>
    <a routerLink="/admin/routes" routerLinkActive="bg-slate-700 text-white"
class="rounded px-3 py-1.5 text-slate-300 hover:bg-slate-800">Approvals</a>
    <a routerLink="/admin/operator-requests" routerLinkActive="bg-slate-700
text-white" class="rounded px-3 py-1.5 text-slate-300
hover:bg-slate-800">Operators</a>
    <a routerLink="/admin/config" routerLinkActive="bg-slate-700 text-white"
class="rounded px-3 py-1.5 text-slate-300 hover:bg-slate-800">Config</a>
}
```

- The html file is not separated in some cases ie not a separate file fro template ,instead it directly embeds html code in the template which is a bad practice as it is prone to errors and intellisense doesn't recognise it as it consider it just a string.

```
import { CommonModule } from '@angular/common';
```

```

import { Component, OnInit, signal } from '@angular/core';
import { ApiService } from '../../core/api.service';

@Component({
  selector: 'app-admin-bookings-page',
  standalone: true,
  imports: [CommonModule],
  template: `
    <section>
      <div class="flex items-center justify-between">
        <div>
          <h1 class="text-2xl font-bold text-white">System Bookings</h1>
          <p class="mt-1 text-sm text-slate-400">All bookings across the entire
platform.</p>
        </div>
      </div>

      @if (error()) {
        <p class="mt-4 rounded bg-rose-900/40 px-3 py-2 text-sm
text-rose-200">{{ error() }}</p>
      }

      <div class="mt-8 overflow-x-auto rounded-xl border border-slate-800
bg-slate-900">
        <table class="w-full text-left text-sm text-slate-300">
          <thead class="border-b border-slate-800 bg-slate-950 text-xs uppercase
text-slate-400">
            <tr>
              <th class="px-4 py-3">Booking ID</th>
              <th class="px-4 py-3">User</th>
              <th class="px-4 py-3">Bus/Route</th>
              <th class="px-4 py-3">Travel Date</th>
              <th class="px-4 py-3">Amount</th>
              <th class="px-4 py-3">Status</th>
              <th class="px-4 py-3">Created At</th>
            </tr>
          </thead>
          <tbody>
            @if (loading()) {
              <tr>
                <td colspan="7" class="px-4 py-8 text-center
text-cyan-300">Loading bookings...</td>

```

```

        </tr>
      } @else if (bookings().length === 0) {
        <tr>
          <td colspan="7" class="px-4 py-8 text-center text-slate-500">No
bookings found.</td>
        </tr>
      } @else {
        @for (b of bookings(); track b.bookingId) {
          <tr class="border-b border-slate-800 last:border-0
hover:bg-slate-800/50">
            <td class="px-4 py-3 font-mono text-[10px] text-slate-400">{{
b.bookingId }}</td>
            <td class="px-4 py-3">
              <div class="flex flex-col">
                <span class="font-medium text-white">{{ b.userName
}}</span>
                <span class="text-[10px] text-slate-500">{{ b.userEmail
}}</span>
              </div>
            </td>
            <td class="px-4 py-3">
              <div class="flex flex-col">
                <span class="text-cyan-300 font-medium">{{
b.registrationNumber }}</span>
                <span class="text-[10px]">{{ b.source }} → {{
b.destination }}</span>
              </div>
            </td>
            <td class="px-4 py-3">{{ b.travelDate }}</td>
            <td class="px-4 py-3 font-medium text-white">₹{{ b.totalAmount
}}</td>
            <td class="px-4 py-3">
              <span class="rounded px-2 py-0.5 text-xs"
[ngClass]="{
                'bg-emerald-900/50 text-emerald-300': b.status ===
'CONFIRMED',
                'bg-amber-900/50 text-amber-300': b.status ===
'PENDING',
                'bg-rose-900/50 text-rose-300':
b.status.includes('CANCELLED')
              }">
                {{ b.status }}

```

```

        </span>
      </td>
      <td class="px-4 py-3 text-slate-400">{{ b.createdAt |
date:'short' }}</td>
    </tr>
  }
}
</tbody>
</table>
</div>
</section>
,
})
export class AdminBookingsPageComponent implements OnInit {
  readonly bookings = signal<any[]>([]);
  readonly loading = signal(true);
  readonly error = signal('');

  constructor(private readonly api: ApiService) {}

  ngOnInit() {
    this.api.getAllBookings().subscribe({
      next: (res) => {
        this.bookings.set(res.data);
        this.loading.set(false);
      },
      error: (err) => {
        this.error.set(err?.error?.errors?.[0] ?? 'Failed to load bookings.');
        this.loading.set(false);
      }
    });
  }
}

```

Integration with server:

- Uses observable subscribe pattern

```

constructor(private readonly api: ApiService) {}

ngOnInit() {
  this.api.getAllBookings().subscribe({
    next: (res) => {

```

```

        this.bookings.set(res.data);
        this.loading.set(false);
    },
    error: (err) => {
        this.error.set(err?.error?.errors?.[0] ?? 'Failed to load bookings.');
        this.loading.set(false);
    }
});
}

```

Uses next,error but not complete

- Dedicated Api service was created but with no modularity ie all api functions were all in one apiservice file.

```

searchBuses(source: string, destination: string, date: string) {
    return
    this.http.get<ApiResponse<BusSearchResult[]>>(`${this.base}/buses/search`, {
        params: { source, destination, date }
    });
}

getPlaces(query?: string, limit = 100) {
    return this.http.get<ApiResponse<string[]>>(`${this.base}/places`, {
        params: { q: query ?? '', limit }
    });
}

getMyBookings() {
    return
    this.http.get<ApiResponse<BookingSummary[]>>(`${this.base}/bookings/my`);
}

createOperatorRequest() {
    return
    this.http.post<ApiResponse<OperatorRequestItem>>(`${this.base}/operator-requests`, {});
}

getMyOperatorRequests() {
    return
    this.http.get<ApiResponse<OperatorRequestItem[]>>(`${this.base}/operator-requests/my`);
}

```



```
getOperatorSummary() {  
    return  
this.http.get<ApiResponse<OperatorSummary>>(`${this.base}/operator/summary`);  
}
```

And many more api functions...